PROJECT PLAN

AMO-Hostaly

Hotel Room Booking System

Software Engineering Course — A.A. 2026/2027

Professor Angelo Gargantini — Dr. Silvia Bonfanti

University of Bergamo

Team:

Ahmed Sabry Azab Mohamed (1100748),

Mohamed Zayan (1092257),

Omar Nassar (1088703).

Version 1.1 — November 2026

1-Introduzione

Il presente documento costituisce il Project Plan ufficiale per lo sviluppo del sistema software Hotel Room Booking System. Il progetto e sviluppato per il corso di Ingegneria del Software (A.A. 2026/2027) sotto la supervisione dei docenti di riferimento.

Scopo e Obiettivi del Sistema: L'obiettivo primario è la progettazione e realizzazione di un'applicazione web robusta in Java destinata alla gestione delle prenotazioni di un hotel. Per garantire una copertura completa delle esigenze operative, il sistema è strutturato per offrire **tre esperienze utente distinte** dedicate a tre attori (Actors) specifici:

1 **Cliente (Client):** Interfaccia per la ricerca delle camere, visualizzazione delle disponibilità e consultazione delle proprie fatture o riepiloghi di prenotazione.

2 **Gestore/Receptionist (Staff):** Dashboard operativa quotidiana per gestire il check-in e check-out, cambiare lo stato delle camere (es. da "pulire" a "disponibile") e addebitare servizi extra.

3 **Amministratore (Admin):** Pannello di controllo riservato alla direzione per la configurazione dell'hotel, la definizione o modifica delle tariffe stagionali, l'aggiunta di nuove camere e l'analisi delle statistiche di occupazione.

2-Modello di Processo

Il modello di processo adottato per lo sviluppo del progetto e un approccio **Agile Scrum** adattato all ambito accademico, basato su uno sviluppo iterativo ed incrementale. La scelta del modello agile consente di mitigare i rischi di integrazione, raffinare progressivamente la modellazione UML e garantire una consegna continua di incrementi di codice funzionante e testato.

Sprint	Arco Temporale	Attivita Principali & Milestones	Deliverable Prodotto
Sprint 1: Analisi&Setup	01/11/2026 - 14/11/2026	Elicitazione requisiti (IEEE 830), Use Case e Class Diagram. Setup repository GitHub, progetto Maven (Spring Boot/Vaadin) in Eclipse.	Project Plan, Schema StarUML, pom.xml.
Sprint 2: Architettura & Core	15/11/2026 - 28/11/2026	Forward Engineering, implementazione del Database locale, logica di backend e sviluppo delle tre interfacce Web (Client, Gestore, Admin) con Vaadin.	Codice sorgente Java compilabile, UI base collegate al database, log4j2.xml.
Sprint 3: Testing & Release	29/11/2026 - 31/12/2026	Rifinitura procedure di Check-in/Check-out, suite di test JUnit 5, Code Review, assemblaggio documentazione finale.	Relazione finale PDF, test JUnit superati, Tag Release v1.0.0.

3- Organizzazione del Progetto

Il team (Gruppo AMO) è composto da 3 membri con responsabilità ben definite:

Membro del Team	Ruolo Principale	Responsabilità Specifiche (Backend & UI & Diagrammi UML)
Ahmed Sabri	Requirements Manager & Gestore Dev	Sviluppo del pannello **Gestore/Staff** (Logica check-in/out, stato camere).
Mohamed Zayan	SCM, Quality & Client UI Dev	Analisi requisiti (IEEE 830), Sviluppo dell'interfaccia **Cliente** (Ricerca camere) .
Omar Nassar	Lead Architect & Admin Dev	Gestione GitHub (Issues, PR), Sviluppo del pannello **Admin** (Gestione tariffe, configurazione).

4. Standard, Linee Guida e Procedure

Per garantire l'omogeneità del codice e la manutenibilità del progetto, il team adotta i seguenti standard vincolanti:

1. **Convenzioni di Codifica Java:** Rispetto delle *Oracle Java Code Conventions* (nomi delle classi in NounCamelCase, metodi in verbCamelCase, costanti in UPPER_SNAKE_CASE, ed information hiding con attributi private e metodi getter/setter).
2. **Standard Obbligatorio di Logging (Log4j):** Divieto assoluto dell'uso di System.out.println. Tutte le informazioni di tracciamento ed errore devono utilizzare **Apache Log4j 2** (org.apache.logging.log4j.Logger e LogManager.getLogger(...)).
3. **Gestione delle Dipendenze con Maven:** Tutte le librerie esterne (JUnit 5, Log4j) sono gestite unicamente tramite il file pom.xml.
4. **Struttura delle Directory e File di Configurazione Eclipse:** Il codice deve risiedere all'interno della cartella codice/AMO_Hostaly/. **Il file .project DEVE essere incluso nel repository e NON deve figurare nel file .gitignore**, per permettere l'importazione diretta in Eclipse.
5. **Regole di Gitignore:** Il file .gitignore deve escludere i file binari generati (.class), la cartella target/ di Maven ed i file temporanei locali del workspace.

5. Attività Gestionali

Le attività gestionali sono strutturate per bilanciare qualità, tempi e collaborazione:

- **Sprint Planning & Task Assignment:** All'inizio di ogni iterazione, i requisiti vengono trasformati in task tecnici assegnati specificamente per lo sviluppo del Backend o di una delle 3 interfacce Web.
- **Kanban Board (GitHub Projects):** Tracciamento visivo e rigoroso delle attività tramite colonne: *To Do, In Progress, Code Review, Done*. Ogni task è collegato a un GitHub Issue ufficiale.
- **Incontri di Sincronizzazione (Stand-up):** Meeting bisettimanali per verificare lo stato di avanzamento delle UI, risolvere blocchi tecnici (es. integrazione tra Vaadin e Spring) e coordinare i merge.
- **Code Review Obbligatoria:** Nessun membro può effettuare un merge diretto sul branch `main`. Ogni modifica richiede una *Pull Request* approvata da almeno un altro membro del team.

6. Rischi Potenziali

In accordo con i principi di Ingegneria del Software, il team ha identificato preventivamente i rischi tecnologici ed organizzativi, definendo le relative strategie di mitigazione:

Identificativo Rischio	Livello d'Impatto	Descrizione del Rischio	Strategia di Mitigazione
R1: Conflitti di Merge Git	🟡 Medio	Conflitti generati da modifiche concorrenti sugli stessi file del codice Java o sul modello StarUML.	Uso rigoroso dei Feature Branch separati (feature/nome-task), commit atomici e frequenti operazioni di git pull.
R2: Ritardi sulla Pianificazione	🔴 Alto	Rallentamenti nella fase di modellazione o nello sviluppo dei test JUnit che minacciano la scadenza.	Monitoraggio costante della Kanban Board.
R3: Incompatibilità Environment	🟢 Basso	Errori nell'importazione del progetto su computer diversi da quelli degli sviluppatori.	Eliminazione di qualsiasi percorso assoluto nel codice, uso di database embedded e configurazione Maven portabile.
R4: framework Spring Boot e Vaadin	🟡 Medio	Curva di apprendimento per i framework Spring Boot e Vaadin che causa ritardi.	Suddivisione netta delle interfacce; supporto reciproco e monitoraggio costante sulla Kanban Board.

7. Personale ed Sforzo Stimato

Lo sforzo complessivo stimato è di **90 - 120 ore-uomo** totali (30-40 ore a persona), ripartito per coprire lo sviluppo full-stack dell'applicazione web.

Fase del Ciclo di Vita	Percentuale Sforzo	Ore-Uomo Stimate (Totale)	Distribuzione per Membro
Modellazione UML (StarUML) & Requisiti	25%	22 - 30 ore	7 - 10 ore / persona
Sviluppo Backend (Spring Boot & DB Locale)	25%	22 - 30 ore	7 - 10 ore
Sviluppo Frontend Web (Vaadin - 3 UI)	30%	27 - 36 ore	9 - 12 ore
SCM, Code Review & Testing Automatizzato (JUnit 5)	20%	18 - 24 ore	6 - 8 ore
TOTALE	**100%**	**90 - 120 ore**	**30 - 40 ore / persona**

8. Metodi e Tecniche

Il progetto si avvale delle seguenti metodologie e strumenti d'avanguardia approvati dalle specifiche del corso:

- **Modellazione Concettuale e Visiva:** Utilizzo di **StarUML 5.1.0** per la creazione di tutti i diagrammi UML prescritti (Use Case, Class, State Machine, Sequence, Activity, Component, Package) e per la generazione del codice Java.
- **Ambiente di Sviluppo (IDE) e Build System:** Sviluppo in ambiente **Eclipse IDE** con automazione della build e gestione dipendenze tramite **Apache Maven** (file pom.xml).
- **Linguaggio di Programmazione: Java SE** sfruttando i principi fondamentali dell'Object-Oriented Programming (Incapsulamento, Ereditarietà, Polimorfismo, Interfacce, Classi Astratte, Enums, Collection Framework).
- **Software Configuration Management (SCM):** Utilizzo di **Git e GitHub** seguendo il workflow ufficiale *GitHub Flow* (Feature Branches, Pull Requests, Code Reviews, e Kanban Project Board).
- **Testing e Tracciamento Log:** Framework **JUnit 5** per lo unit testing automatizzato ed **Apache Log4j 2** per la registrazione strutturata degli eventi di runtime.
- **Architettura Web Application**: Utilizzo di Spring Boot per la logica di business e la configurazione del server embedded.
- **Interfaccia Utente (Frontend):** Utilizzo del framework **Vaadin**, che permette la creazione delle tre interfacce web (Admin, Gestore, Cliente) interamente in linguaggio Java, mantenendo il paradigma a oggetti senza dipendere da HTML/JS esterni.

- **Gestione Dati (Database):** Utilizzo di un **Database relazionale Embedded/Locale** (es. H2 Database). I dati vengono salvati localmente all'interno del progetto

9. Garanzia di Qualità

La gestione della qualità copre sia la qualità del *Prodotto Software* (manutenibilità, leggibilità, assenza di difetti finanziari) sia la qualità del *Processo di Sviluppo*:

- **Verifica e Validazione (V&V):** La *Verifica* persegue la correttezza del codice rispetto alla progettazione UML. La *Validazione* assicura che il sistema soddisfi appieno le esigenze operative dell'hotel (gestione corretta del check-in, totale fattura corretto, gestione dello stato della camera).
- **Revisione del Codice (Code Review):** Nessun commit viene fuso direttamente sul branch main senza l'approvazione di almeno un altro membro del team tramite Pull Request.
- **Obiettivo di Code Coverage:** Garanzia di una copertura di test superiore all'**80%** sulle classi di core business logic (Reservation, Invoice, HotelManager).

10. Pacchetti di Lavoro / WBS (Work Packages)

La scomposizione gerarchica delle attività del progetto (Work Breakdown Structure - WBS) è articolata nei seguenti pacchetti di lavoro:

Codice WBS	Nome Pacchetto di Lavoro	Descrizione Dettagliata Attività	Responsabile Assegnato
WP 1.1	Project Plan	Redazione Project Plan, Setup GitHub.	AHMED
WP 2.1	Modellazione StarUML	Creazione Use Case, Class Diagram, State Machine e Sequence Diagrams (.mdj).	Tutto il Team
WP 3.1	Setup Spring/Vaadin & Maven	Inizializzazione progetto Maven, configurazione DB Locale e Log4j, Specifica IEEE 830..	MOHAMED
WP 4.1	Core Backend & DB	Entità principali (Room, Guest, Reservation). E Implementazione classi Guest, Check-in/out, Room, RoomStatus, RoomType e logica persistenza.	Tutto il Team
WP 5.1	Interfaccia Gestore	Sviluppo UI Staff con Vaadin (Dashboard camere, Check-in/out).	AHMED

WP 5.2	Interfaccia Admin	Sviluppo UI Direzione (Admin, Gestione tariffe stagionali, statistiche).	OMAR
WP 5.3	Interfaccia Cliente	Sviluppo UI Pubblica (Cliente, Ricerca camere disponibili, visualizzazione).	MOHAMED
WP 6.1	JUnit & Testing	Stesura ed esecuzione test case automatici sulle classi core.	OMAR
WP 7.1	Release & Documenti	Assemblaggio PDF finale, Code Review conclusiva, Tag su GitHub.	Tutto il Team

11. Risorse (Resources)

Le risorse hardware, software ed infrastrutturali impiegate per il progetto comprendono:

- **Hardware:** Personal Computer dei membri del team (architettura x86_64, SO Windows / macOS / Linux).
- **Ambiente di Sviluppo (IDE):** Eclipse IDE for Java Developers (versione 2026-09 o equivalente).
- **Software di Modellazione:** StarUML 5.1.
- **Build Tool e Repository Locale:** Apache Maven 3.x integrato in Eclipse.
- **Librerie esterne e Dipendenze:** JUnit 5 (junit-jupiter-api e junit-jupiter-engine v5.11.0) ed Apache Log4j 2 (log4j-api e log4j-core v2.26.1).
- **Framework Web & Server: Spring Boot** (include web server Apache Tomcat integrato) e **Vaadin Flow** per la generazione dell'interfaccia.
- **Database: Embedded DB (SQLite)** collegato tramite JDBC/Spring Data, con salvataggio locale dei file dati.
- **Infrastruttura Cloud e SCM:** Piattaforma GitHub (Repository remoto, Issue Tracking, Pull Requests, e GitHub Project Board (**Kanban Board**)).

12. Budget e Programma (Budget and Schedule)

Trattandosi di un progetto accademico, il budget economico è valorizzato unicamente in termini di **Ore-Uomo di Sviluppo** (pari a 90 - 120 ore totali per il team). La pianificazione temporale si articola su un arco di circa 6 settimane:

Milestone / Scadenza	Data Target	Attività di riferimento e Criterio di Successo
Milestone 1: Project Plan Submitted	[14/11/2026]	Condivisione del Project Plan via Issue GitHub.
Milestone 2: Design & Model Approved	[28/11/2026]	Completamento di tutti i diagrammi UML in StarUML e generazione dello scheletro del codice.
Milestone 3: Core Implementation Done	[05/12/2026]	Completamento delle classi del modello, motore di calcolo fatture ed integrazione di Log4j.
Milestone 4: Test Suite & Freeze Code	[31/12/2026]	Superamento del 100% dei test JUnit 5, congelamento repository almeno 5 giorni prima dell'orale.

13. Gestione delle Modifiche (Changes)

In un progetto software i cambiamenti sono inevitabili. Per evitare il caos organizzativo (scope creep), le modifiche vengono gestite attraverso la seguente:

1. **Apertura dell'Issue:** Qualsiasi proposta di modifica dei requisiti, refactoring o segnalazione di bug viene registrata come un nuovo *Issue* su GitHub.
2. **Valutazione d'Impatto:** Il team valuta l'impatto della modifica sui diagrammi UML di StarUML, sul codice Java e sui test JUnit esistenti.
3. **Sviluppo su Feature Branch:** La modifica viene implementata in isolamento su un branch dedicato (feature/issue-#) (es. feature/fix-admin-login).
4. **Verifica ed Approvazione:** Il codice viene testato con JUnit 5. Quando i test hanno esito positivo, viene aperta una *Pull Request* che deve essere approvata da un peer prima del merge sul main.

14. Consegna (Delivery)

La procedura formale di consegna del progetto al docente si compone delle seguenti fasi obbligatorie:

1. **Approvazione del Project Plan (1 Mese Prima):** Il presente documento viene sottomesso tramite apertura di un Issue sul repository GitHub assegnato a @garganti e @silviabonfanti per ottenerne l'approvazione formale.
2. **Congelamento del Repository (5 Giorni Prima):** Almeno 5 giorni prima della data della presentazione orale, l'intero codice sorgente (progetto Eclipse/Maven AMO_Hostaly), il file StarUML (.mdj) sul repository GitHub.
3. **Verifiche del Repository:** verifichiamo la presenza del file .project (non inserito nel gitignore), l'assenza di percorsi assoluti, la presenza del file pom.xml con le dipendenze JUnit 5 e Log4j, e lo storico reale dei commit condotti dai singoli membri.
4. **Presentazione Orale**